

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA05

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 20%

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

1. Real-Time Face Detection and Recognition Pipeline

A system must process a live video stream to:

- A. Capture frames from a camera
- B. Detect faces in each frame
- C. Recognize faces using a pre-trained dataset
- D. Display bounding boxes and labels in real time

Constraints:

- Must operate continuously
- Should minimize processing delay
- Should handle multiple faces

Write detailed pseudocode for the complete pipeline using OpenCV concepts, including image acquisition, preprocessing, detection, recognition, and display. Ensure modular design and efficient looping structure.

2. OCR Pipeline for Document Processing

A document processing system must:

- A. Read input images containing printed text
- B. Enhance image quality (noise removal, contrast improvement)
- C. Segment text regions
- D. Extract and display recognized text

Constraints:

- Input images may be noisy and low contrast
- Batch processing required

Write pseudocode for an end-to-end OCR pipeline using OpenCV functions. Clearly structure preprocessing, segmentation, and text extraction stages.

3. Motion Detection and Tracking System

A surveillance system must:

- Read video input
- Detect motion between consecutive frames
- Track moving objects
- Highlight detected motion regions

Constraints:

- Should handle dynamic background
- Must operate in near real-time

Design pseudocode for the system, including frame differencing, filtering, tracking logic, and visualization. Ensure proper control flow and handling of continuous video streams.

4. Vehicle Counting System using Video

A traffic monitoring system must:

- Read video frames
- Detect vehicles entering a region of interest (ROI)
- Track movement across frames
- Count vehicles and display count

Constraints:

- Avoid double counting
- Handle multiple vehicles simultaneously

Write pseudocode for the complete system, including detection, tracking, counting logic, and visualization using drawing functions.

5. Facial Expression Recognition System

A mobile-based system must:

- Capture images from a camera
- Detect faces
- Extract facial features
- Classify expressions (happy, sad, neutral)
- Display results

Constraints:

- Limited computational resources
- Real-time response required

Write pseudocode for the pipeline, ensuring efficient processing and modular design for detection, feature extraction, and classification.

6. Real-Time Object Detection Pipeline

A system must:

- A. Capture video frames
- B. Detect objects using a trained model
- C. Draw bounding boxes and labels
- D. Display results in real time

Constraints:

- Must process frames efficiently
- Handle multiple objects

Write detailed pseudocode with modular steps for detection, visualization, and efficient looping.

7. Image Difference Detection System

A system must:

- A. Read two images
- B. Compare them
- C. Highlight differences

Constraints:

- Handle noise
- Ensure accurate difference detection

Write pseudocode including preprocessing, comparison logic, and visualization.

8. Video Frame Extraction System

A system must:

- A. Read video input
- B. Extract frames at fixed intervals
- C. Save selected frames

Constraints:

- Efficient storage
- Avoid redundant frames

Write structured pseudocode for frame selection and saving logic.

9. Background Subtraction Pipeline

A surveillance system must:

- A. Read video stream
- B. Separate foreground from background
- C. Highlight moving objects

Constraints:

- Handle dynamic background
- Reduce noise

Write pseudocode including background modeling, subtraction, filtering, and display.

10. Edge Detection Pipeline for Real-Time Video

A system must:

- A. Capture video frames
- B. Apply edge detection
- C. Display results

Constraints:

- Real-time processing
- Maintain clarity of edges

Write pseudocode with preprocessing, edge detection, and visualization modules.

11. Multi-Image Batch Processing System

A system must:

- A. Read multiple images from a folder
- B. Apply preprocessing and feature detection
- C. Store results

Constraints:

- Batch processing
- Efficient memory usage

Write pseudocode with modular structure and looping logic.

12. Face Detection with ROI Extraction

A system must:

- A. Detect faces in an image
- B. Extract face regions

C. Save cropped faces

Constraints:

- Handle multiple faces
- Avoid redundant processing

Write pseudocode including detection, ROI extraction, and saving logic.

13. Video Stabilization Pipeline

A system must:

- A. Read video frames
- B. Detect motion between frames
- C. Stabilize output

Constraints:

- Maintain real-time processing
- Reduce jitter

Write pseudocode including motion estimation, transformation, and display.

14. Text Detection in Natural Scene Images

A system must:

- A. Read images
- B. Detect text regions
- C. Highlight detected areas

Constraints:

- Handle varying backgrounds
- Improve detection accuracy

Write pseudocode including preprocessing, region detection, and visualization.

15. Feature Matching Pipeline

A system must:

- A. Read two images
- B. Extract features
- C. Match features
- D. Display matches

Constraints:

- Robust to scale and rotation

- Efficient matching

Write pseudocode with modular steps for detection, matching, and visualization.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear

		follow			
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Computer Vision – Detailed Pseudocode

Questions 1, 2, 3, 5 and 9

1. Real-Time Face Detection and Recognition Pipeline

Objective: Process a live video stream, detect multiple faces, recognize registered users, and display bounding boxes and labels continuously with minimum delay.

Pseudocode:

```
BEGIN

INITIALIZE camera
LOAD pre-trained face detection model
LOAD pre-trained face recognition dataset/model

SET recognition_threshold
SET display window

WHILE camera is open:

    CAPTURE frame from camera

    IF frame cannot be captured:
        BREAK

    RESIZE frame to suitable resolution
    CONVERT frame from BGR to grayscale
    APPLY optional noise reduction

    DETECT all faces in the frame

    FOR each detected face:

        EXTRACT face region
        RESIZE face to recognition model input size
        NORMALIZE face image if required

        RECOGNIZE face using pre-trained model

        IF confidence is acceptable:
            label = recognized person's name
        ELSE:
            label = "Unknown"

        DRAW bounding box around face
        DISPLAY label and confidence

    DISPLAY processed frame

    IF user presses 'q':
        BREAK

RELEASE camera
CLOSE all windows

END
```

Modular Design: main() → initialize_camera() → load_models() → capture_frame() → preprocess_frame() → detect_faces() → recognize_face() → draw_results() → display_frame()

Efficiency: Resize frames, use grayscale for detection, recognize only detected face regions, skip frames when necessary, use lightweight models, and release resources correctly.

Conclusion: The modular pipeline continuously detects and recognizes multiple faces while reducing unnecessary processing and display delay.

2. OCR Pipeline for Document Processing

Objective: Read printed text from noisy and low-contrast images, enhance them, segment text regions, perform OCR, and support batch processing.

Pseudocode:

```
BEGIN

GET list of all input images from folder

FOR each image in input folder:

    READ image using OpenCV

    IF image cannot be read:
        CONTINUE to next image

    RESIZE image if resolution is low
    CONVERT image to grayscale

    APPLY median/Gaussian filtering
        → Remove noise

    APPLY CLAHE or histogram equalization
        → Improve contrast

    APPLY adaptive thresholding
        → Separate text from background

    APPLY morphological operations
        → Remove small noise
        → Connect broken characters if required

    FIND contours/text regions
    EXTRACT text regions

    PASS processed image to OCR engine
    EXTRACT recognized text

    DISPLAY recognized text
    SAVE text to output file

END FOR

DISPLAY completion message

END
```

Pipeline: Input Image → Resize → Grayscale → Noise Removal → Contrast Enhancement → Adaptive Thresholding → Morphological Processing → Text Segmentation → OCR → Recognized Text

Stage	OpenCV Concept / Purpose
Read image	cv2.imread()
Resize	cv2.resize() – improves small text visibility
Grayscale	cv2.cvtColor() – simplifies processing
Noise removal	cv2.GaussianBlur() / cv2.medianBlur()
Contrast	cv2.createCLAHE() – improves local contrast
Thresholding	cv2.adaptiveThreshold() – handles uneven illumination
Morphology	cv2.morphologyEx() – removes noise / improves characters
Segmentation	cv2.findContours() – identifies text regions

Batch Processing: The FOR loop processes every image in the input folder and saves the corresponding OCR result automatically.

Conclusion: Enhancement and adaptive segmentation make the OCR pipeline more reliable for low-quality documents, while batch looping supports large document collections.

3. Motion Detection and Tracking System

Objective: Read video, detect motion between frames, track moving objects, and highlight motion regions while operating near real time.

Pseudocode:

```
BEGIN

OPEN video source
READ first frame

CONVERT first frame to grayscale
APPLY Gaussian blur

INITIALIZE previous frame
INITIALIZE object tracker

WHILE video is available:

    READ current frame

    IF frame is unavailable:
        BREAK

    RESIZE frame for real-time processing
    CONVERT current frame to grayscale
    APPLY Gaussian blur

    COMPUTE frame difference:
        difference = current_frame - previous_frame

    APPLY threshold to difference

    APPLY morphological opening
        → Remove noise

    APPLY morphological dilation
        → Strengthen motion regions

    FIND contours

    FOR each contour:

        CALCULATE contour area

        IF area > minimum_motion_area:

            CALCULATE bounding box
            MATCH object with existing tracker
            UPDATE object position

            DRAW bounding box
            DISPLAY "Motion Detected"

    UPDATE tracker
    SET previous_frame = current_frame

    DISPLAY frame with tracking results

    IF user presses 'q':
        BREAK

RELEASE video source
CLOSE windows

END
```

Dynamic Background: Simple frame differencing can react to trees, rain and shadows. For better robustness, replace or supplement frame differencing with MOG2 background subtraction.

Tracking Logic: If a detected object matches an existing object, update its position; otherwise create a new track. Keep a temporary track when an object disappears briefly and remove it after a longer absence.

Real-Time Optimization: Resize frames, use grayscale images, ignore very small contours, use efficient OpenCV operations, and process only a region of interest where appropriate.

Conclusion: Frame differencing, filtering, contour detection and object tracking provide continuous motion detection while maintaining near-real-time performance.

5. Facial Expression Recognition System

Objective: Capture camera frames, detect faces, extract facial features, classify expressions as Happy, Sad or Neutral, and display the result under limited computational resources.

Pseudocode:

```
BEGIN

INITIALIZE mobile camera

LOAD lightweight face detector
LOAD trained expression classification model

SET expressions = {Happy, Sad, Neutral}

WHILE camera is active:

    CAPTURE frame
    RESIZE frame to lower resolution
    CONVERT frame to grayscale

    DETECT faces

    FOR each detected face:

        EXTRACT face region
        RESIZE face to model input size
        NORMALIZE pixel values

        EXTRACT facial features
        CLASSIFY expression using trained model

        IF predicted class is valid:
            expression = Happy / Sad / Neutral
        ELSE:
            expression = Unknown

        DRAW bounding box
        DISPLAY expression label

    DISPLAY result frame

    IF user exits:
        BREAK

RELEASE camera
CLOSE windows

END
```

Modular Design: main() → initialize_camera() → load_face_detector() → load_expression_model() → capture_frame() → detect_face() → extract_features() → classify_expression() → display_result()

Feature Extraction: Features may include eye position, eyebrow position, mouth shape, nose position, and facial landmark distances. A trained classifier maps these features to Happy, Sad or Neutral.

Resource Optimization: Resize frames, use grayscale where appropriate, process only detected face regions, use lightweight models, and skip frames if necessary.

Conclusion: A lightweight modular pipeline can provide real-time facial expression recognition on mobile devices while controlling computational cost.

9. Background Subtraction Pipeline

Objective: Read a video stream, separate foreground moving objects from the background, reduce noise, and highlight detected objects.

Pseudocode:

```
BEGIN

OPEN video stream

INITIALIZE background subtractor
  Example: MOG2

CREATE morphological kernel

WHILE video stream is active:

  READ frame

  IF frame cannot be read:
    BREAK

  RESIZE frame if necessary

  APPLY background subtraction
  GENERATE foreground mask

  REMOVE shadows if required

  APPLY morphological opening
    → Remove small noise

  APPLY morphological closing
    → Fill small gaps

  APPLY dilation
    → Strengthen foreground objects

  FIND contours in foreground mask

  FOR each contour:

    CALCULATE area

    IF area > minimum threshold:

      FIND bounding rectangle
      DRAW bounding box
      LABEL as moving object

  DISPLAY original frame
  DISPLAY foreground mask

  IF user presses 'q':
    BREAK

RELEASE video stream
CLOSE all windows

END
```

Pipeline: Video Stream → Frame Acquisition → Background Modeling → Background Subtraction → Foreground Mask → Thresholding → Morphological Filtering → Contour Detection → Moving Object Detection → Bounding Box → Display

Dynamic Background Handling: MOG2 maintains a statistical model of the background and adapts to gradual changes such as moving tree branches, changing shadows and other environmental motion.

Noise Reduction: Morphological opening removes small isolated noise, closing fills small gaps, and dilation strengthens detected foreground regions.

Operation	OpenCV Function
Video input	cv2.VideoCapture()
Background modeling	cv2.createBackgroundSubtractorMOG2()
Thresholding	cv2.threshold()
Noise filtering	cv2.morphologyEx()
Contour detection	cv2.findContours()
Bounding box	cv2.boundingRect()

Operation	OpenCV Function
Display	cv2.imshow()

Conclusion: MOG2 combined with thresholding and morphological filtering provides effective foreground detection in dynamic environments while reducing noise.